

API Security Risk Analysis Report

Submitted By: ABIGAEL .C.RONO

Date: March 12, 2026

1. Executive Summary	3
2.Scope of Assessment	3
3.Tools Used	4
3.1. Postman	4
3.2. Web Browser Developer Tools	4
3.3. Public API Documentation	5
4. Detailed Findings	6
4.1 Endpoint: /users	6
4.2 Endpoint: /posts	7
4.3 Endpoint: /comments	8
4.4 Todos Endpoint (/todos)	10
5. Summary & Recommendations	11

1. Executive Summary

This report evaluates the security posture of the JSONPlaceholder API, a publicly accessible demo API. Although designed for testing and learning purposes, the assessment treats the API as a production system to highlight common security risks. The analysis focuses on:

- Open or unauthenticated endpoints
- Excessive data exposure
- Weak or missing authentication tokens
- Authorization issues
- Missing rate limiting
- Input validation issues

Key findings indicate that: all endpoints are open, unauthenticated, and susceptible to excessive data exposure. Recommendations include enforcing authentication, input validation, and rate limiting to mitigate potential risks in production environments.

2.Scope of Assessment

The assessment covered the following endpoints:

Endpoint	Method	Description
----------	--------	-------------

/users	GET	Fetch all users
/users/{id}	GET	Fetch single user
/posts	GET / POST	Fetch or create posts
/posts/{id}	GET	Fetch single post
/comments	GET / POST	Fetch or create comments
/todos	GET / PUT	Fetch or update todos

Testing was limited to **non-destructive requests**, focusing solely on **risk identification**. No exploitation or data manipulation was performed beyond the API's demo functionality.

3.Tools Used

Tools Used for the Assessment

The following tools were used to perform the API security assessment:

3.1. Postman

Postman was used to send HTTP requests to the API endpoints and analyze responses. It allowed testing of various HTTP methods such as GET, POST, and PUT, as well as inspection of response headers and payloads.

Purpose in the assessment:

- Sending API requests
- Inspecting response data
- Testing authentication requirements
- Identifying excessive data exposure
- Observing input validation behavior

3.2. Web Browser Developer Tools

Browser developer tools were used to observe network requests and inspect HTTP headers.

Purpose in the assessment:

- Viewing request and response headers
- Inspecting API responses

- Observing HTTP status codes

3.3. Public API Documentation

The official JSONPlaceholder documentation was used to understand available endpoints and expected responses.

API documentation:

<https://jsonplaceholder.typicode.com>

Purpose in the assessment:

- Reviewing available endpoints
- Understanding API structure
- Determining expected request formats

4. Detailed Findings

4.1 Endpoint: /users

- Method: GET
- URL: <https://jsonplaceholder.typicode.com/users>
- Authentication: None required
- Headers: Optional `Accept: application/json`

Identified Risks:

- Open endpoint → unauthenticated access
- Excessive data exposure → full user details including emails, addresses, phone numbers
- Missing rate limiting

Severity: High

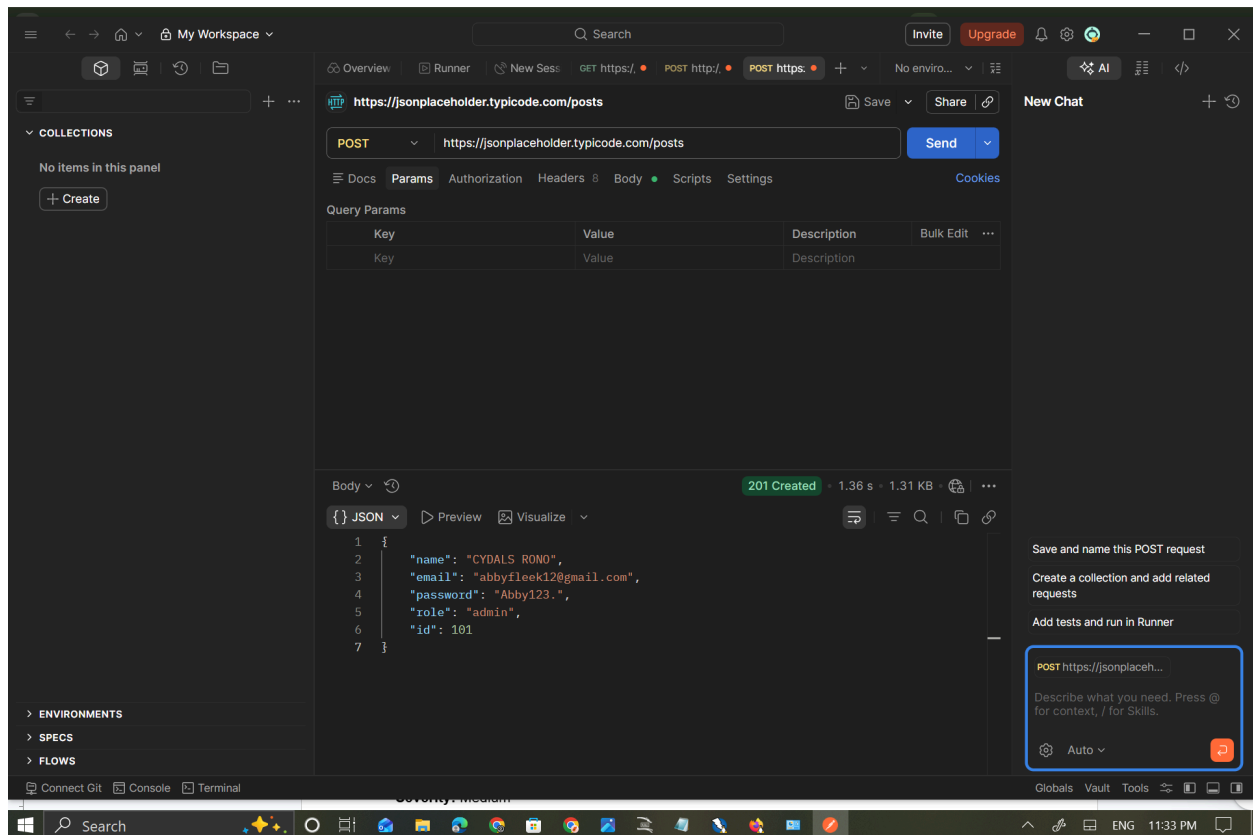
Recommendations:

- Require authentication (API key / OAuth2 / JWT)
- Limit returned fields to essential information only
- Implement rate limiting

Severity: High

Recommendations:

- Enforce authentication for creating/updating posts
- Validate payloads to prevent injection attacks
- Limit returned data fields



4.3 Endpoint: /comments

- Method: GET / POST
- URL: `https://jsonplaceholder.typicode.com/comments`

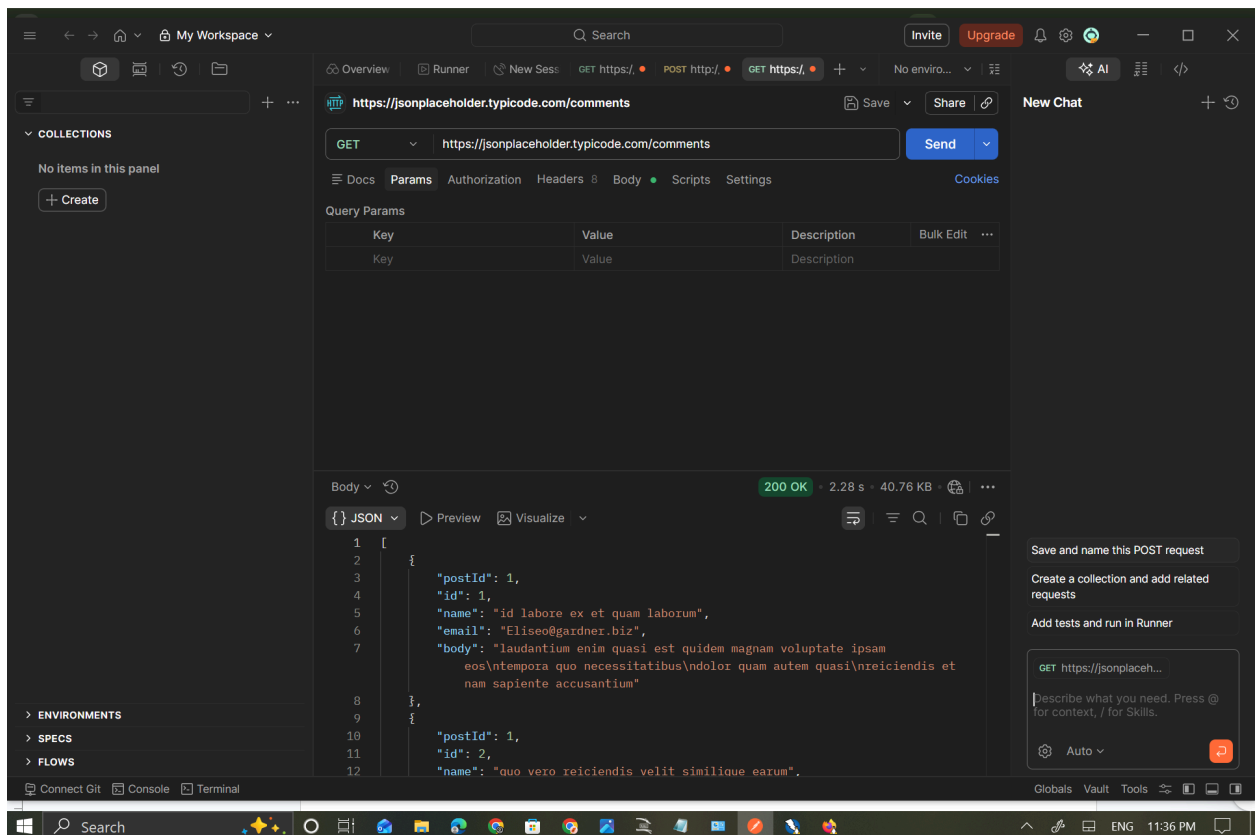
Identified Risks:

- Open endpoint → no authentication
- Excessive data exposure → email, name, postId included
- Missing input validation

Severity: Medium

Recommendations:

- Require authentication for POST requests
- Sanitize and validate inputs
- Limit returned information



4.4 Todos Endpoint (/todos)

Requests: GET /todos, PUT /todos/{id}

Authentication Required: None

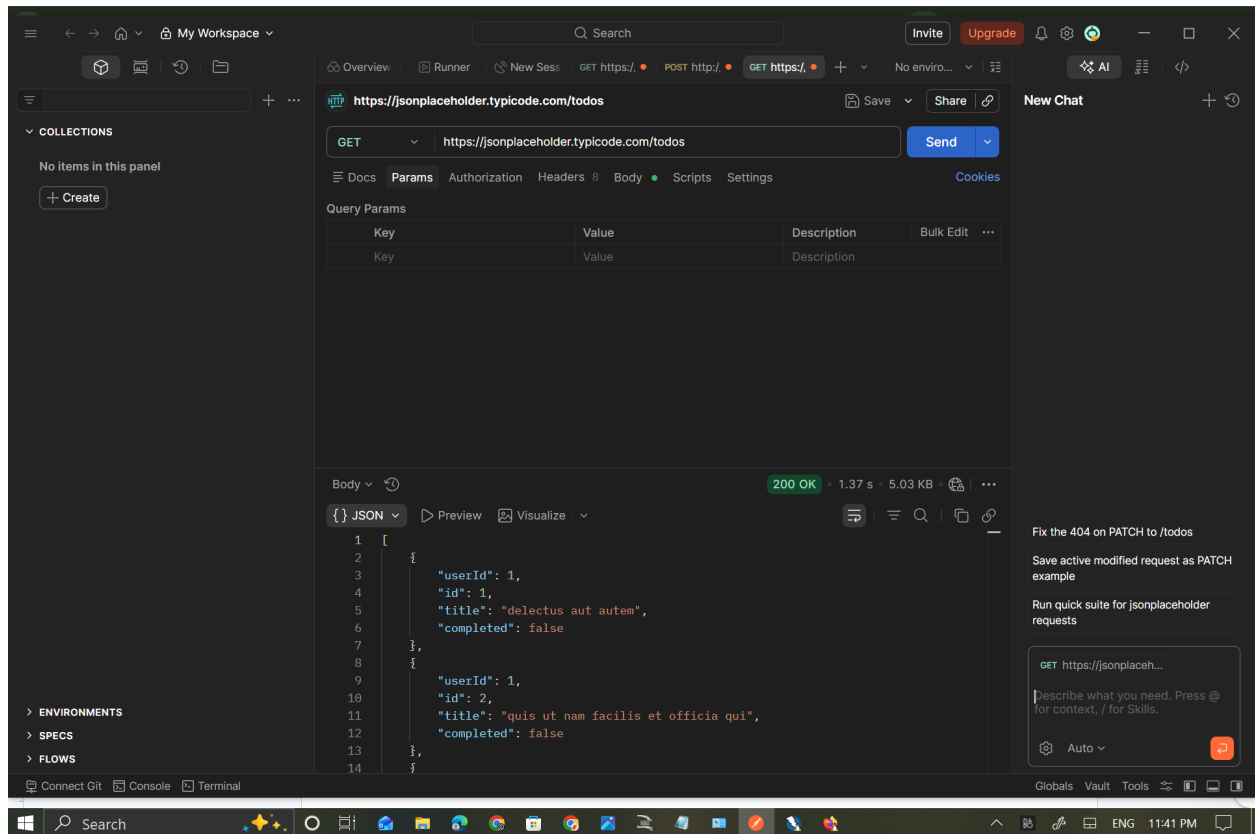
Identified Risks:

- Open access → anyone can view or update todos
- No authorization checks
- Missing rate limiting

Severity: High

Recommendations:

- Implement authentication and authorization checks
- Limit request rate
- Validate inputs before updating



5. Summary & Recommendations

- Even demo APIs highlight typical risks in production APIs.
- Authentication, authorization, rate limiting, and input validation are **must-have controls**.
- Excessive data exposure should be minimized; never return more than required.
- Always classify risk severity (High/Medium/Low) based on potential impact.